SPPU Third Year IT – Operating Systems

Practical Programs – Complete C and Shell Code

2024 Pattern | Practicals 1–8 | CO1–CO5

This PDF contains the source code for all eight Operating Systems laboratory assignments. Practical 2 includes separate demonstrations for fork/wait, zombie and orphan processes. Practical 6 contains two independent FIFO programs.

Important: Compile and run the C programs on Linux or a Linux environment such as Ubuntu. The pthread programs require the `-pthread` compiler option. The shell program requires Bash.

Practical	Program
1	Shell Address Book
2A	fork() and wait() – parent/child sorting
2B	Zombie process demonstration
2C	Orphan process demonstration
3	Producer–Consumer using semaphores and mutex
4	Dining Philosophers using semaphores
5	Banker's Algorithm
6A + 6B	Full-duplex FIFO communication
7	Full-duplex parent–child pipe communication
8	SSTF, SCAN and C-LOOK disk scheduling

Practical 1 – Shell Programming: Address Book (CO1)

Menu-driven address book supporting create, view, insert, delete, modify and exit. Each record is stored as name|phone|address. Names are treated as unique identifiers.

Program 1

File: addressbook.sh

Compile/run: `chmod +x addressbook.sh && ./addressbook.sh`

Run with Bash. The script uses awk for exact name-field matching and a temporary file for safe updates.

```
#!/bin/bash

FILE="addressbook.txt"

create_book() {
    : > "$FILE"
    echo "Address book created successfully."
}

view_book() {
    if [ -f "$FILE" ]; then
        if [ -s "$FILE" ]; then
            echo "NAME | PHONE | ADDRESS"
            cat "$FILE"
        else
            echo "Address book is empty."
        fi
    else
        echo "Address book does not exist. Create it first."
    fi
}

insert_record() {
    if [ ! -f "$FILE" ]; then
        echo "Address book does not exist. Create it first."
        return
    fi

    read -r -p "Enter name: " name
    read -r -p "Enter phone: " phone
    read -r -p "Enter address: " address

    # Reject duplicate names (exact first-field match).
    if awk -F'|' -v n="$name" '$1 == n { found=1 } END { exit !found }' "$FILE"; then
        echo "A record with this name already exists."
        return
    fi

    printf '%s|%s|%s\n' "$name" "$phone" "$address" >> "$FILE"
    echo "Record inserted successfully."
}

delete_record() {
    if [ ! -f "$FILE" ]; then
        echo "Address book does not exist."
        return
    fi

    read -r -p "Enter exact name to delete: " name
    if ! awk -F'|' -v n="$name" '$1 == n { found=1 } END { exit !found }' "$FILE"; then
        echo "Record not found."
        return
    fi

    awk -F'|' -v n="$name" '$1 != n' "$FILE" > "${FILE}.tmp" &&
        mv "${FILE}.tmp" "$FILE"
    echo "Record deleted successfully."
}

modify_record() {
    if [ ! -f "$FILE" ]; then
        echo "Address book does not exist."
        return
    fi

    read -r -p "Enter exact name to modify: " old_name
    if ! awk -F'|' -v n="$old_name" '$1 == n { found=1 } END { exit !found }' "$FILE"; then
        echo "Record not found."
        return
    fi

    read -r -p "Enter new name: " new_name
    read -r -p "Enter new phone: " new_phone
    read -r -p "Enter new address: " new_address
```

```
    awk -F'|' -v old="$old_name" -v nn="$new_name" \
        -v np="$new_phone" -v na="$new_address" \
        'BEGIN { OFS="|" }
         $1 == old { print nn, np, na; next }
         { print $1, $2, $3 }' "$FILE" > "${FILE}.tmp" &&
        mv "${FILE}.tmp" "$FILE"

    echo "Record modified successfully."
}

while true
do
    echo
    echo "------ ADDRESS BOOK ------"
    echo "1. Create address book"
    echo "2. View address book"
    echo "3. Insert a record"
    echo "4. Delete a record"
    echo "5. Modify a record"
    echo "6. Exit"
    read -r -p "Enter your choice: " choice

    case "$choice" in
        1) create_book ;;
        2) view_book ;;
        3) insert_record ;;
        4) delete_record ;;
        5) modify_record ;;
        6) echo "Exiting..."; exit 0 ;;
        *) echo "Invalid choice. Try again." ;;
    esac
done
```

Practical 2 – Process Management (CO2)

Program 2A – fork(), wait() and sorting in parent and child

Program 2A

File: process_sort.c

Compile/run: `gcc process_sort.c -o process_sort && ./process_sort`

The parent and child have separate logical address spaces after fork(). Their sorting changes do not overwrite each other's array.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void bubble_sort(int a[], int n, int ascending)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            int swap_needed = ascending
                ? (a[j] > a[j + 1])
                : (a[j] < a[j + 1]);

            if (swap_needed)
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

int main(void)
{
    int a[100], n;

    printf("Enter number of integers (1-100): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 100)
    {
        printf("Invalid number of integers.\n");
        return 1;
    }

    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }
    else if (pid == 0)
    {
        // Child sorts its own copy in ascending order.
        bubble_sort(a, n, 1);
        printf("\nChild PID: %d\nAscending order: ", getpid());
        for (int i = 0; i < n; i++)
            printf("%d ", a[i]);
        printf("\n");
        exit(0);
    }
    else
    {
        int status;

        // Parent sorts its own copy in descending order.
        bubble_sort(a, n, 0);
        printf("\nParent PID: %d\nDescending order: ", getpid());
        for (int i = 0; i < n; i++)
            printf("%d ", a[i]);
        printf("\n");

        waitpid(pid, &status, 0);
        printf("Child process completed.\n");
    }

    return 0;
}
```

Program 2B – Zombie process demonstration

Program 2B

File: zombie.c

Compile/run: `gcc zombie.c -o zombie && ./zombie`

During the parent's sleep, inspect the process state in another terminal. A terminated child may appear with state Z until the parent reaps it.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child (PID %d) is exiting.\n", getpid());
        exit(0);
    }
    else
    {
        printf("Parent (PID %d) created child %d.\n", getpid(), pid);
        printf("Parent will sleep for 20 seconds without wait().\n");
        printf("During this time, inspect the child state using: ps -el\n");

        sleep(20);

        printf("Parent is now calling wait().\n");
        waitpid(pid, NULL, 0);
        printf("Child has been reaped; zombie entry is cleared.\n");
    }

    return 0;
}
```

Program 2C – Orphan process demonstration

Program 2C

File: orphan.c

Compile/run: `gcc orphan.c -o orphan && ./orphan`

The child continues after its parent exits and is adopted by PID 1 or another configured subreaper. The displayed adopted PID depends on the environment.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Original parent PID: %d\n", getppid());

        sleep(5);

        printf("After original parent exits:\n");
        printf("Current parent PID: %d\n", getppid());
        printf("Child is still running as an orphan.\n");
    }
    else
    {
        printf("Parent PID %d is exiting.\n", getpid());
        exit(0);
```

```
    }
    return 0;
}
```

Practical 3 – Producer–Consumer using Semaphores and Mutex (CO3)

Program 3 – Bounded buffer with one producer and one consumer

Program 3

File: producer_consumer.c

Compile/run: `gcc producer_consumer.c -o producer_consumer -pthread && ./producer_consumer`

empty_slots starts at SIZE, full_slots starts at 0. The mutex protects buffer indices and data. sem_wait/sem_post are POSIX semaphore operations.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5
#define ITEMS 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty_slots, full_slots;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    (void)arg;

    for (int i = 1; i <= ITEMS; i++)
    {
        int item = i * 10;

        sem_wait(&empty_slots);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 1; i <= ITEMS; i++)
    {
        int item;

        sem_wait(&full_slots);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);

        sleep(2);
    }

    return NULL;
}

int main(void)
{
    pthread_t p, c;

    sem_init(&empty_slots, 0, SIZE);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
```

```
    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);

    printf("Producer-Consumer demonstration completed.\n");
    return 0;
}
```

Practical 4 – Dining Philosophers Problem (CO3, CO4)

Program 4

File: dining_philosophers.c

Compile/run: `gcc dining_philosophers.c -o dining_philosophers -pthread && ./dining_philosophers`

The room semaphore is initialized to N-1, which breaks the circular-wait condition and prevents deadlock in this solution. It does not guarantee starvation freedom.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
#define MEALS 3

sem_t forks[N];
sem_t room;

void *philosopher(void *arg)
{
    int id = *(int *)arg;

    for (int meal = 0; meal < MEALS; meal++)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        // At most N-1 philosophers may compete for forks.
        sem_wait(&room);

        sem_wait(&forks[id]);              // Left fork
        sem_wait(&forks[(id + 1) % N]);    // Right fork

        printf("Philosopher %d is eating.\n", id);
        sleep(1);

        printf("Philosopher %d finished eating.\n", id);

        sem_post(&forks[(id + 1) % N]);
        sem_post(&forks[id]);
        sem_post(&room);
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[N];
    int ids[N];

    sem_init(&room, 0, N - 1);

    for (int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);

    for (int i = 0; i < N; i++)
    {
        ids[i] = i;
        pthread_create(&threads[i], NULL, philosopher, &ids[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < N; i++)
        sem_destroy(&forks[i]);

    sem_destroy(&room);

    printf("All philosophers finished.\n");
    return 0;
}
```

Practical 5 – Banker's Algorithm (CO4)

Program 5

File: bankers_algorithm.c

Compile/run: `gcc bankers_algorithm.c -o bankers_algorithm && ./bankers_algorithm`

This program performs the safety check for the supplied Allocation, Max and Available data. An unsafe result means no safe sequence was found.

```
#include <stdio.h>

#define MAX 10

int main(void)
{
    int n, m;
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int available[MAX], work[MAX], finish[MAX] = {0};
    int safe[MAX], count = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);

    if (n < 1 || n > MAX || m < 1 || m > MAX)
    {
        printf("Invalid dimensions. Maximum is %d x %d.\n", MAX, MAX);
        return 1;
    }

    printf("Enter Allocation Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix (%d x %d):\n", n, m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available vector (%d values):\n", m);
    for (int j = 0; j < m; j++)
        scanf("%d", &available[j]);

    // Calculate Need = Max - Allocation.
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (max[i][j] < alloc[i][j])
            {
                printf("Invalid input: Max cannot be less than Allocation.\n");
                return 1;
            }
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }

    for (int j = 0; j < m; j++)
        work[j] = available[j];

    // Safety algorithm.
    while (count < n)
    {
        int found = 0;

        for (int i = 0; i < n; i++)
        {
            if (finish[i] == 0)
            {
                int j;
                for (j = 0; j < m; j++)
                {
                    if (need[i][j] > work[j])
                        break;
                }
```

```
                if (j == m)
                {
                    for (int k = 0; k < m; k++)
                        work[k] += alloc[i][k];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if (!found)
            break;
    }

    if (count == n)
    {
        printf("\nSystem is in SAFE state.\nSafe sequence: ");
        for (int i = 0; i < n; i++)
            printf("P%d%s", safe[i], (i == n - 1) ? "\n" : " -> ");
    }
    else
    {
        printf("\nSystem is in UNSAFE state; no safe sequence was found.\n");
    }

    return 0;
}
```

Practical 6 – Full-Duplex Communication using FIFOs (CO5)

Program 6A – First independent process: input and display

Program 6A

File: fifo_process1.c

Compile/run: `gcc fifo_process1.c -o fifo_process1 && ./fifo_process1`

Run Process 1 in one terminal and Process 2 in another terminal, from the same directory. Process 1 creates the FIFO names.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>
#include <errno.h>

#define FIFO1 "fifo1"
#define FIFO2 "fifo2"
#define SIZE 4096

int main(void)
{
    char input[SIZE] = "";
    char line[512];
    char result[SIZE];
    size_t used = 0;
    int fd, n;

    if (mkfifo(FIFO1, 0666) == -1 && errno != EEXIST)
    {
        perror("mkfifo fifo1");
        return 1;
    }
    if (mkfifo(FIFO2, 0666) == -1 && errno != EEXIST)
    {
        perror("mkfifo fifo2");
        return 1;
    }

    printf("Enter sentences (type END on a line by itself):\n");

    while (fgets(line, sizeof(line), stdin) != NULL)
    {
        if (strcmp(line, "END\n") == 0 || strcmp(line, "END") == 0)
            break;

        size_t len = strlen(line);
        if (used + len >= sizeof(input))
        {
            printf("Input limit reached; remaining input ignored.\n");
            break;
        }

        memcpy(input + used, line, len);
        used += len;
        input[used] = '\0';
    }

    // Send all input to Process 2.
    fd = open(FIFO1, O_WRONLY);
    if (fd == -1)
    {
        perror("open fifo1");
        return 1;
    }

    size_t sent = 0;
    while (sent < used)
    {
        n = write(fd, input + sent, used - sent);
        if (n <= 0)
        {
            perror("write fifo1");
            close(fd);
            return 1;
        }
        sent += (size_t)n;
    }
    close(fd);

    // Receive report from Process 2.
    fd = open(FIFO2, O_RDONLY);
    if (fd == -1)
    {
```

```
        perror("open fifo2");
        return 1;
    }

    n = read(fd, result, sizeof(result) - 1);
    if (n > 0)
    {
        result[n] = '\0';
        printf("\nReport received from Process 2:\n%s", result);
    }
    else
    {
        printf("No report received.\n");
    }
    close(fd);

    unlink(FIFO1);
    unlink(FIFO2);
    return 0;
}
```

Program 6B – Second independent process: count and return report

Program 6B

File: fifo_process2.c

Compile/run: `gcc fifo_process2.c -o fifo_process2 && ./fifo_process2`

Start Process 1 first, then Process 2. Character count excludes newline characters but includes spaces and punctuation. Lines count newline-terminated lines plus a final unterminated line.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>
#include <ctype.h>

#define FIFO1 "fifo1"
#define FIFO2 "fifo2"
#define SIZE 4096

int main(void)
{
    char text[SIZE];
    char report[512];
    int fd, n;
    size_t total = 0;
    int characters = 0, words = 0, lines = 0, in_word = 0;

    // Receive all input until Process 1 closes FIFO1.
    fd = open(FIFO1, O_RDONLY);
    if (fd == -1)
    {
        perror("open fifo1");
        return 1;
    }

    while (total < sizeof(text) - 1 &&
           (n = read(fd, text + total, sizeof(text) - 1 - total)) > 0)
    {
        total += (size_t)n;
    }
    close(fd);
    text[total] = '\0';

    // Count characters (excluding newline), words and lines.
    for (size_t i = 0; i < total; i++)
    {
        unsigned char ch = (unsigned char)text[i];

        if (ch != '\n')
            characters++;

        if (ch == '\n')
            lines++;

        if (isspace(ch))
            in_word = 0;
        else if (!in_word)
        {
            words++;
            in_word = 1;
        }
    }

    if (total > 0 && text[total - 1] != '\n')
```

```
            lines++;

        // Write report to a text file.
        FILE *fp = fopen("result.txt", "w");
        if (fp == NULL)
        {
            perror("fopen result.txt");
            return 1;
        }

        fprintf(fp, "Number of Characters: %d\n", characters);
        fprintf(fp, "Number of Words: %d\n", words);
        fprintf(fp, "Number of Lines: %d\n", lines);
        fclose(fp);

        // Read report file.
        fp = fopen("result.txt", "r");
        if (fp == NULL)
        {
            perror("fopen result.txt");
            return 1;
        }

        size_t report_len = fread(report, 1, sizeof(report) - 1, fp);
        report[report_len] = '\0';
        fclose(fp);

        // Send report to Process 1.
        fd = open(FIFO2, O_WRONLY);
        if (fd == -1)
        {
            perror("open fifo2");
            return 1;
        }

        size_t sent = 0;
        while (sent < report_len)
        {
            n = write(fd, report + sent, report_len - sent);
            if (n <= 0)
            {
                perror("write fifo2");
                close(fd);
                return 1;
            }
            sent += (size_t)n;
        }
        close(fd);

        printf("Analysis completed. Report saved in result.txt\n");
        return 0;
    }
```

Practical 7 – Full-Duplex Communication using Unnamed Pipes (CO5)

Program 7

File: full_duplex_pipe.c

Compile/run: `gcc full_duplex_pipe.c -o full_duplex_pipe && ./full_duplex_pipe`

Create a text file first, then enter its path. Two unnamed pipes are used because each ordinary pipe is unidirectional.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>

#define NAME_SIZE 256
#define BUF_SIZE 1024

int main(void)
{
    int parent_to_child[2];
    int child_to_parent[2];
    pid_t pid;
    char filename[NAME_SIZE];

    if (pipe(parent_to_child) == -1 || pipe(child_to_parent) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid > 0)
    {
        // Parent: send filename, receive file contents.
        close(parent_to_child[0]);
        close(child_to_parent[1]);

        printf("Enter file path: ");
        if (fgets(filename, sizeof(filename), stdin) == NULL)
            filename[0] = '\0';

        filename[strcspn(filename, "\n")] = '\0';

        size_t len = strlen(filename) + 1;
        if (write(parent_to_child[1], filename, len) != (ssize_t)len)
            perror("write filename");

        close(parent_to_child[1]);

        printf("\nFile contents:\n");
        char buffer[BUF_SIZE];
        ssize_t n;

        while ((n = read(child_to_parent[0], buffer, sizeof(buffer))) > 0)
        {
            if (write(STDOUT_FILENO, buffer, (size_t)n) == -1)
            {
                perror("write stdout");
                break;
            }
        }

        close(child_to_parent[0]);
        waitpid(pid, NULL, 0);
    }
    else
    {
        // Child: receive filename, open file, send contents.
        close(parent_to_child[1]);
        close(child_to_parent[0]);

        if (read(parent_to_child[0], filename, sizeof(filename)) <= 0)
        {
            close(parent_to_child[0]);
            close(child_to_parent[1]);
```

```
            _exit(1);
        }
        close(parent_to_child[0]);

        int file_fd = open(filename, O_RDONLY);
        if (file_fd == -1)
        {
            char message[] = "Error: unable to open the requested file.\n";
            write(child_to_parent[1], message, sizeof(message) - 1);
        }
        else
        {
            char buffer[BUF_SIZE];
            ssize_t n;

            while ((n = read(file_fd, buffer, sizeof(buffer))) > 0)
            {
                ssize_t offset = 0;
                while (offset < n)
                {
                    ssize_t written = write(child_to_parent[1],
                                            buffer + offset,
                                            (size_t)(n - offset));
                    if (written <= 0)
                    {
                        perror("write pipe");
                        break;
                    }
                    offset += written;
                }
                if (offset < n)
                    break;
            }

            close(file_fd);
        }

        close(child_to_parent[1]);
        _exit(0);
    }

    return 0;
}
```

Practical 8 – Disk Scheduling: SSTF, SCAN and C-LOOK (CO5)

Program 8

File: disk_scheduling.c

Compile/run: `gcc disk_scheduling.c -o disk_scheduling && ./disk_scheduling`

For the sample queue 82, 170, 43, 140, 24, 16, 190; head 50; disk size 200; direction high: SSTF = 208, SCAN = 332, C-LOOK = 341 cylinders. SCAN counts movement to the physical boundary; C-LOOK counts the circular jump.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

void sort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void SSTF(const int a[], int n, int head)
{
    int done[MAX] = {0};
    int total = 0;

    printf("\nSSTF sequence: %d", head);

    for (int i = 0; i < n; i++)
    {
        int best = -1;
        int min_distance = 2147483647;

        for (int j = 0; j < n; j++)
        {
            int distance = abs(head - a[j]);
            if (!done[j] && distance < min_distance)
            {
                min_distance = distance;
                best = j;
            }
        }

        done[best] = 1;
        total += min_distance;
        head = a[best];
        printf(" -> %d", head);
    }

    printf("\nSSTF total head movement = %d cylinders\n", total);
}

void SCAN(const int a[], int n, int head, int disk_size, int direction)
{
    int b[MAX], total = 0, current = head;

    for (int i = 0; i < n; i++)
        b[i] = a[i];
    sort(b, n);

    int split = 0;
    while (split < n && b[split] < head)
        split++;

    printf("\nSCAN sequence: %d", head);

    if (direction == 1)  // Initially moving towards higher cylinders.
    {
        for (int i = split; i < n; i++)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }

        if (current != disk_size - 1)
        {
```

```
            total += abs(current - (disk_size - 1));
            current = disk_size - 1;
            printf(" -> %d", current);
        }

        for (int i = split - 1; i >= 0; i--)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }
    }
    else  // Initially moving towards lower cylinders.
    {
        for (int i = split - 1; i >= 0; i--)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }

        if (current != 0)
        {
            total += current;
            current = 0;
            printf(" -> %d", current);
        }

        for (int i = split; i < n; i++)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }
    }

    printf("\nSCAN total head movement = %d cylinders\n", total);
}

void CLOOK(const int a[], int n, int head, int direction)
{
    int b[MAX], total = 0, current = head;

    for (int i = 0; i < n; i++)
        b[i] = a[i];
    sort(b, n);

    int split = 0;
    while (split < n && b[split] < head)
        split++;

    printf("\nC-LOOK sequence: %d", head);

    if (direction == 1)  // Initially moving towards higher cylinders.
    {
        for (int i = split; i < n; i++)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }

        if (split > 0)
        {
            total += abs(current - b[0]); // Count circular jump.
            current = b[0];
            printf(" -> %d", current);

            for (int i = 1; i < split; i++)
            {
                total += abs(current - b[i]);
                current = b[i];
                printf(" -> %d", current);
            }
        }
    }
    else  // Initially moving towards lower cylinders.
    {
        for (int i = split - 1; i >= 0; i--)
        {
            total += abs(current - b[i]);
            current = b[i];
            printf(" -> %d", current);
        }

        if (split < n)
        {
            total += abs(current - b[n - 1]); // Count circular jump.
            current = b[n - 1];
            printf(" -> %d", current);
```

```
            for (int i = n - 2; i >= split; i--)
            {
                total += abs(current - b[i]);
                current = b[i];
                printf(" -> %d", current);
            }
        }
    }

    printf("\nC-LOOK total head movement = %d cylinders\n", total);
}

int main(void)
{
    int n, head, disk_size, direction;
    int requests[MAX];

    printf("Enter number of requests (1-%d): ", MAX);
    scanf("%d", &n);

    if (n < 1 || n > MAX)
    {
        printf("Invalid number of requests.\n");
        return 1;
    }

    printf("Enter request queue:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &requests[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size (number of cylinders): ");
    scanf("%d", &disk_size);

    if (disk_size < 1 || head < 0 || head >= disk_size)
    {
        printf("Invalid disk size or head position.\n");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        if (requests[i] < 0 || requests[i] >= disk_size)
        {
            printf("Request outside disk range.\n");
            return 1;
        }
    }

    printf("Enter initial direction (1 = high, 0 = low): ");
    scanf("%d", &direction);

    if (direction != 0 && direction != 1)
    {
        printf("Invalid direction.\n");
        return 1;
    }

    SSTF(requests, n, head);
    SCAN(requests, n, head, disk_size, direction);
    CLOOK(requests, n, head, direction);

    return 0;
}
```

Compilation and Practical Checklist

Practical	Compile / Run
1	`chmod +x addressbook.sh && ./addressbook.sh`
2A	`gcc process_sort.c -o process_sort && ./process_sort`
2B	`gcc zombie.c -o zombie && ./zombie`
2C	`gcc orphan.c -o orphan && ./orphan`
3	`gcc producer_consumer.c -o producer_consumer -pthread`
4	`gcc dining_philosophers.c -o dining_philosophers -pthread`
5	`gcc bankers_algorithm.c -o bankers_algorithm`
6A	`gcc fifo_process1.c -o fifo_process1`
6B	`gcc fifo_process2.c -o fifo_process2`
7	`gcc full_duplex_pipe.c -o full_duplex_pipe`
8	`gcc disk_scheduling.c -o disk_scheduling`

Practical 6 execution: Open two terminals in the same directory. Run `./fifo_process1` in Terminal 1, then `./fifo_process2` in Terminal 2. Process 1 waits for Process 2 to open FIFO1.

General note: These are educational reference implementations. Always check return values, understand the resource and process behavior, and follow any specific code format prescribed by your lab instructor.